

Indice

Easyfatt Sync — Documentazione tecnica sviluppatori

1. Introduzione progetto

Cos'è Easyfatt Sync

Obiettivo software

Stack tecnologico

2. Architettura applicazione

Diagramma logico

Main process

Renderer process

Preload

IPC (canali principali)

electron-store

Scheduler

Sync engine

Google OAuth

Updater

Backup system

Support system

3. Struttura progetto

File principali (ruolo)

4. Configurazione sviluppo

Prerequisiti

Installazione

OAuth in sviluppo

Avvio in sviluppo

Script build

Variabili ambiente

5. Google OAuth setup

Progetto Google Cloud (Aven Labs)

OAuth Desktop App

Scopes

File credenziali

Token locali

Sicurezza

6. Sistema sincronizzazione

Flusso dati

Watcher file

Sync programmata

Gestione Excel

Google Sheets

Progress UI

Gestione errori

7. Sistema aggiornamenti automatici

Componenti

Comportamento app

Release workflow (sintesi)

Versioning

8. Sistema supporto

Architettura

Payload (campi principali)

Backend di esempio

Email cliente

9. Backup e ripristino

Formato file JSON

Struttura payload

Restore

Backup automatici

10. Sicurezza

Electron hardening

Preload sicuro

Token e credenziali

API backend

Buone pratiche release

11. Workflow release

Checklist completa

Publish con token

Compatibilità aggiornamenti

12. TODO / roadmap

13. Troubleshooting tecnico

OAuth

Sync

Updater

Build

Permessi file

Token Google in backup

Contatti tecnici

Easyfatt Sync — Documentazione tecnica sviluppatori

Documentazione di riferimento per sviluppo, manutenzione, build e release dell'applicazione desktop **Easyfatt Sync** (Aven Labs).

1. Introduzione progetto

Cos'è Easyfatt Sync

Easyfatt Sync è un'applicazione desktop che automatizza l'aggiornamento di un foglio **Google Sheets** a partire dal file **Excel** esportato da **Easyfatt** (tipicamente elenco clienti).

Il cliente finale non configura Google Cloud: usa credenziali OAuth **centralizzate Aven Labs** (`oauth_credentials.json` nel pacchetto) e salva solo il **token personale** in locale.

Obiettivo software

Obiettivo	Descrizione
Affidabilità	Sync idempotente (clear + rewrite foglio), retry lettura Excel su file bloccato
Semplicità	UI a impostazioni, senza terminale
Automazione	Watch file, cron, promemoria, backup automatici

Obiettivo	Descrizione
Manutenibilità	Moduli Node separati, IPC tipizzati via preload
Distribuzione	Installer Windows (NSIS) e DMG macOS, update via GitHub Releases

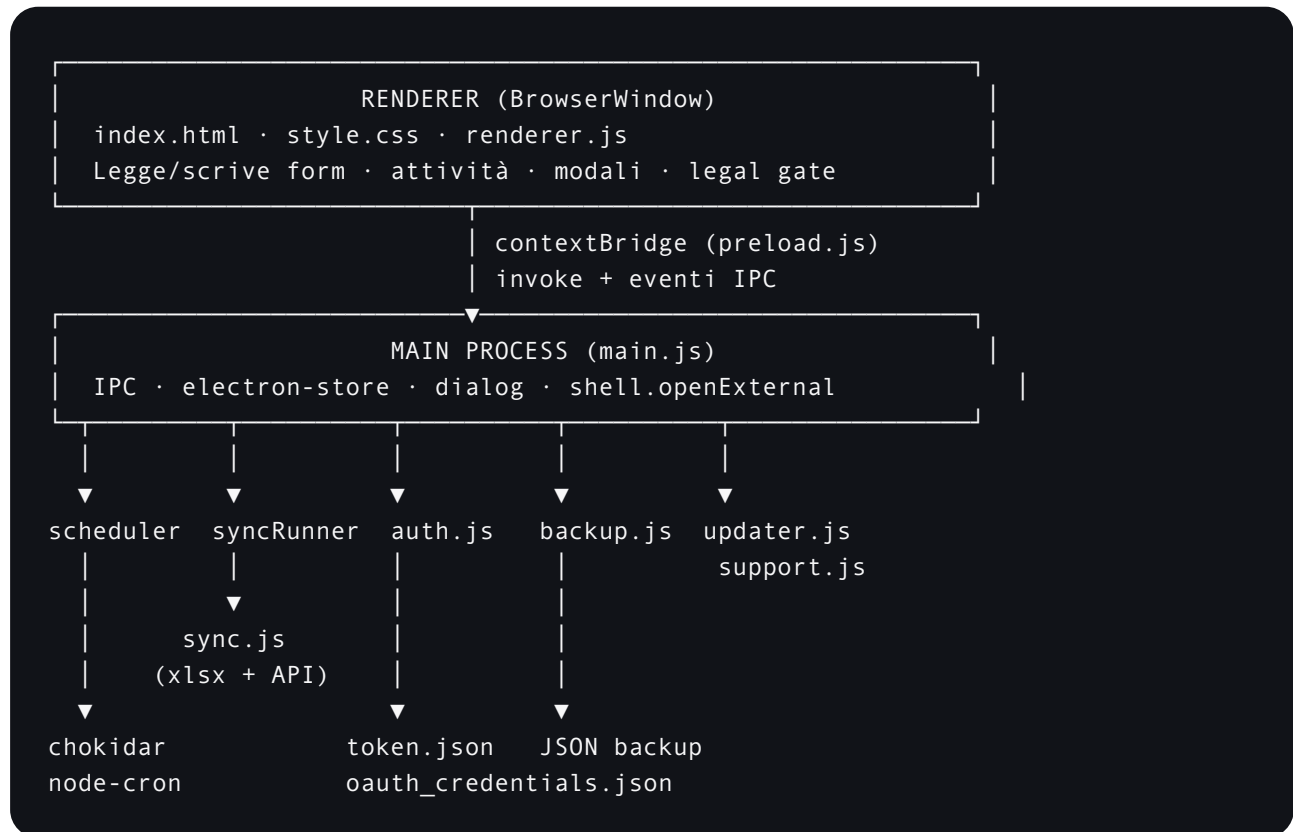
Stack tecnologico

Layer	Tecnologia
Desktop shell	Electron 42.x
Runtime	Node.js (bundled con Electron)
UI	HTML / CSS / JavaScript vanilla (<code>renderer /</code>)
Persistenza config	<code>electron-store</code>
Excel	<code>xlsx</code> (lettura buffer async)
Google	<code>googleapis</code> + OAuth 2.0
File watch	<code>chokidar</code>
Scheduler	<code>node-cron</code>
Build	<code>electron-builder</code>
Auto-update	<code>electron-updater</code> + <code>electron-log</code>
HTTP supporto	<code>fetch</code> nativo verso API Aven Labs

Piattaforme target: Windows 10/11 (x64), macOS (ARM64 e Intel).

2. Architettura applicazione

Diagramma logico



Main process

- Punto di ingresso: `main.js`.
- Crea `BrowserWindow`, registra handler `ipcMain`, orchestra moduli.
- **Non** espone Node al renderer (`nodeIntegration: false`, `contextIsolation: true`).
- All'avvio: carica config, `restartScheduler`, login item, check update (solo packaged).
- All'uscita: `stopScheduler()` su `before-quit`.

Renderer process

- Carica `renderer/index.html` + `renderer.js`.
- Interagisce solo con `window.easyfattySync` (API preload).
- Gestisce: form impostazioni, stato sync, attività recenti (max 50), tema chiaro/scuro, legal gate, supporto, backup UI, aggiornamenti.

Preload

File: `preload.js` .

Espone un sottoinsieme sicuro di operazioni tramite

```
contextBridge.exposeInMainWorld("easyfattSync", { ... }) .
```

Pattern:

- Operazioni request/response → `ipcRenderer.invoke(channel, ...args)` .
- Eventi main → renderer → `ipcRenderer.on("log" | "config-updated" | "update-state" | ...)` .

IPC (canali principali)

Canale	Direzione	Funzione
<code>get-config</code> / <code>save-config</code>	R  M	Lettura/scrittura config
<code>select-excel</code> / <code>select-backup-folder</code>	R → M	Dialog file/cartella
<code>connect-google</code> / <code>logout-google</code> / <code>is-google-authorized</code>	R  M	OAuth
<code>sync-now</code>	R → M	Sync manuale
<code>backup-create</code> / <code>backup-preview</code> / <code>backup-restore</code> / <code>get-backup-meta</code>	R  M	Backup
<code>check-for-updates</code> / <code>download-update</code> / <code>install-update-now</code>	R  M	Updater
<code>submit-support-request</code>	R → M	Supporto
<code>get-legal-status</code> / <code>accept-legal</code>	R  M	Privacy/termini
<code>open-external</code>	R → M	Link HTTPS nel browser di sistema
<code>log</code>	M → R	Stream log attività
<code>config-updated</code>	M → R	

Canale	Direzione	Funzione
		Refresh UI dopo sync/restore
update-state / update-progress	M → R	Stato aggiornamenti

electron-store

- Istanza singleton in `main.js`: `const store = new Store()`.
- Chiavi principali:
 - `config` — oggetto impostazioni utente (vedi `syncState.getDefaultConfig()`).
 - `legalAccepted`, `legalAcceptedAt`, `legalVersion`.
 - `backupLastCreatedAt`, `backupLastRestoredAt`, `backupLastAutomaticAt`.

Scheduler

Modulo: `scheduler.js`.

Responsabilità:

- **File watcher** (`chokidar`) con `awaitWriteFinish` e `debounce 5s`.
- **Sync programmata** (`node-cron`) su orari `syncTimes`.
- **Promemoria** sync mancante (`reminderTimes`).
- **Backup automatici** (cron giornaliero/settimanale/mensile).

Ogni `restartScheduler` esegue `stopScheduler()` (chiude watcher, ferma cron, cancella timeout) prima di riavviare.

Mutex sync: `syncRunner.runSync` usa flag `syncInProgress` condiviso tra manuale, watch e cron.

Sync engine

Pipeline:

1. `sync.js` — legge Excel (async + retry), prepara matrice valori, chiama Google Sheets API.
2. `syncRunner.js` — orchestrazione, notifiche, `recordSyncSuccess` su store.
3. `sheetRange.js` — range dinamico colonne (non più `A:Z` fisso).
4. `errors.js` — messaggi client-friendly.

Google OAuth

Modulo: `auth.js` .

- Credenziali app: `oauth_credentials.json` (root progetto, **non** in git).
- Token utente: `%APPDATA%/EasyfattSync/token.json` (Windows) o `~/EasyfattSync/token.json` (macOS/Linux).
- Flusso: server HTTP locale temporaneo su `127.0.0.1` , browser esterno, callback, salvataggio token.
- Cache client OAuth + listener `tokens` per persistere refresh.

Updater

Modulo: `updater.js` .

- Attivo solo se `app.isPackaged` .
- `autoDownload: false` , `autoInstallOnAppQuit: false` — l'utente conferma download e install.
- Publish configurato verso GitHub (`package.json` → `build.publish`).
- Eventi verso renderer su canale `update-state` .

Backup system

Modulo: `backup.js` .

- Export JSON impostazioni + stato legale + token Google opzionale.
- Restore con validazione `backupVersion` .
- Rotazione backup automatici per cartella e retention N.
- Lock `backupInProgress` per evitare backup concorrenti.

Support system

Moduli: `support.js` , `supportConstants.js` , `supportIssueTypes.js` .

- POST JSON a `https://aven-labs.com/api/support/easyfatt-sync` .
- Timeout 30s, validazione lato client e server (esempio Next.js in `backend-examples/`).

3. Struttura progetto

```
easyfatt-sync-app/
├─ main.js           # Entry Electron, IPC, lifecycle
├─ preload.js        # Bridge sicuro renderer ↔ main
├─ auth.js           # OAuth Google, token file
├─ sync.js           # Excel → Google Sheets
├─ syncRunner.js     # Wrapper sync + mutex + notifiche
├─ syncState.js      # Default config, merge, validazione scheduler
├─ scheduler.js      # Watch, cron, backup automatico
├─ backup.js         # Backup/restore JSON
├─ updater.js        # electron-updater
├─ support.js        # Invio richiesta supporto
├─ supportConstants.js # URL API, email
├─ supportIssueTypes.js # Etichette tipo problema
├─ notifications.js  # Notifiche desktop native
├─ loginSettings.js  # Avvio con il sistema (openAtLogin)
├─ errors.js         # Messaggi errore UI
├─ sheetRange.js     # Helper range Sheets
├─ legalConstants.js # Versione legale, URL policy
├─ package.json      # Scripts, electron-builder, publish
├─ UPDATES.md        # Note release/updater
├─ assets/
│   └─ icon.png      # Icona app / build
├─ renderer/
│   ├── index.html
│   ├── style.css
│   └─ renderer.js
├─ legal/
│   ├── privacy-easyfatt-sync.md
│   └─ terms-easyfatt-sync.md
├─ backend-examples/ # API supporto (Next.js + Resend)
└─ docs/             # Questa documentazione
```

File principali (ruolo)

File	Ruolo
main.js	Window, IPC, collegamento moduli, restartScheduler
preload.js	API window.easyfattSync
auth.js	OAuth loopback, read/write token, cache client

File	Ruolo
<code>sync.js</code>	Lettura XLSX, clear/update Sheets
<code>syncRunner.js</code>	Mutex sync, notifiche successo/errore
<code>scheduler.js</code>	Automazioni temporizzate e watch file
<code>backup.js</code>	Payload backup, restore, cleanup
<code>updater.js</code>	Check/download/install release GitHub
<code>support.js</code>	Validazione form + fetch API
<code>renderer/renderer.js</code>	Logica UI, attività, form, modali

4. Configurazione sviluppo

Prerequisiti

- **Node.js** 18+ (consigliato LTS allineato a Electron 42).
- **npm** 9+.
- Account Google Cloud (solo per generare `oauth_credentials.json` di test).
- Per build macOS: macOS + Xcode CLI tools.
- Per build Windows: Windows (o CI) per NSIS.

Installazione

```
git clone <repository-url>
cd easyfatt-sync-app
npm install
```

OAuth in sviluppo

Copia il file credenziali OAuth Desktop nella root (vedi **\$5**):

```
easyfatt-sync-app/oauth_credentials.json # gitignored
```

Il token si crea al primo “Collega Google” nell'app:

```
~/EasyfattSync/token.json # macOS/Linux
%APPDATA%/EasyfattSync/token.json # Windows
```

Avvio in sviluppo

```
npm run dev
```

Equivalente a `electron .` — **non** abilita auto-updater reale (messaggio dev in UI aggiornamenti).

Script build

Script	Output
<code>npm run dist:win</code>	Installer NSIS x64 in <code>dist/</code>
<code>npm run dist:mac</code>	DMG ARM64 (Apple Silicon)
<code>npm run dist:mac:intel</code>	DMG x64 (Intel)

Configurazione builder in `package.json` → sezione `"build"` :

- `appId` : `com.avenlabs.easyfattsync`
- `productName` : `Easyfatt Sync`
- `publish` : `GitHub DanielMatei0/easyfatt-sync`

Variabili ambiente

Variabile	Uso
<code>GH_TOKEN</code>	Publish release su GitHub da CI o macchina build (electron-builder)
<code>RESEND_API_KEY</code>	Solo backend supporto (Next.js), non nell'app Electron
<code>APPDATA</code> / <code>HOME</code>	Path token utente (lettura in <code>auth.js</code>)

File **non** committare (vedi `.gitignore`):

- `oauth_credentials.json`
- `token.json`
- `.env` / `.env.local`

5. Google OAuth setup

Progetto Google Cloud (Aven Labs)

1. Console **Google Cloud**.
2. Crea o seleziona un progetto (es. "Aven Labs Easyfatt Sync").
3. Abilita **Google Sheets API**.
4. Crea credenziali **OAuth 2.0 Client ID** di tipo **Desktop app** (consigliato per Electron).

OAuth Desktop App

- Con Desktop app i redirect `http://127.0.0.1:<porta>/callback` sono gestiti senza registrare ogni porta in anticipo (a differenza di alcuni Web client).
- L'app avvia un server locale temporaneo e apre il browser con `shell.openExternal` .

Scopes

```
https://www.googleapis.com/auth/spreadsheets
```

Accesso in lettura/scrittura ai fogli dell'account collegato.

File credenziali

`oauth_credentials.json` (formato Google):

```
{
  "installed": {
    "client_id": "...",
    "client_secret": "...",
    "redirect_uris": ["http://localhost"]
  }
}
```

In produzione il file è **incluso nel pacchetto**; il cliente non crea un progetto Google proprio.

Token locali

- Salvati in `EasyfattSync/token.json` per utente/macchina.
- `access_type: "offline"` + `prompt: "consent"` al primo collegamento per ottenere refresh token.
- Refresh persistito su evento `tokens` del client OAuth.

Sicurezza

Regola	Implementazione
Secret non nel renderer	Solo main process legge credenziali
Token non in backup di default	Checkbox esplicita in UI backup
Revoca	“Disconnetti Google” elimina <code>token.json</code>
Messaggio scadenza	<code>errors.js</code> + refresh fallito → “Ricollega Google”

6. Sistema sincronizzazione

Flusso dati

```
Excel (.xlsx) → xlsx (sheet_to_json) → matrice [header, ...rows]
→ values.clear (range dinamico) → values.update (A1, RAW)
```

- Viene usato il **primo foglio** del workbook Excel (`SheetNames[0]`).
- Il nome foglio Google è `config.sheetName` (default `Clienti`).

Watcher file

- Attivo se `watchEnabled` && `excelPath` .
- `chokidar` con `awaitWriteFinish.stabilityThreshold: 5000` ms.
- Debounce aggiuntivo 5s prima di chiamare `runSync` .
- Skip se sync già in corso o config incompleta (`canRunAutomatedSync`).

Sync programmata

- `scheduleEnabled` + array `syncTimes` (formato `HH:MM`).
- Un job `node-cron` per ogni orario valido.
- Orari non validi ignorati con warning in log.

Gestione Excel

- Lettura **async** (`fs.promises.readFile` + `XLSX.read` `buffer`).
- **Retry** fino a 5 tentativi su `EBUSY` / `EPERM` (file aperto in Easyfatt/Excel).
- Messaggio utente: file in uso → chiudi Easyfatt e riprova.

Google Sheets

- Clear solo sulle colonne effettive (`sheetRange.buildClearRange`).
- Update da `'SheetName'!A1` con tutte le righe.

Progress UI

Il main invia log speciali:

```
__SYNC_PROGRESS__:<percent>:<messaggio>
```

Il renderer aggiorna barra progresso senza inondare l'elenco attività.

Gestione errori

- `syncRunner` cattura errori, notifica desktop (se abilitate), rilancia messaggio friendly.
- Scheduler logga `Errore sincronizzazione: ...` senza crashare il processo.

7. Sistema aggiornamenti automatici

Basato su **electron-updater** e release **GitHub**.

Componenti

File	Ruolo
<code>updater.js</code>	Listener <code>autoUpdater</code> , emit eventi UI
<code>package.json</code> → <code>build.publish</code>	Provider GitHub
<code>dist/latest.yml</code>	Metadati versione (generato da electron-builder)

Comportamento app

- Init solo se `app.isPackaged` .
- Check automatico ~4s dopo avvio.
- Download e install **solo su azione utente**.
- Listener registrati una sola volta (`listenersRegistered`).

Release workflow (sintesi)

Vedi anche `[UPDATES.md] (../../UPDATES.md)` e §11.

1. Bump `version` in `package.json`.
2. Build piattaforma target.
3. Tag Git `vX.Y.Z`.
4. GitHub Release con asset: `.exe` / `.dmg`, `latest.yml`, `.blockmap` se presenti.
5. App installata legge `latest.yml` e propone update.

Versioning

Seguire **Semantic Versioning** per comunicare breaking change agli utenti:

- **PATCH** — bugfix, messaggi, piccole correzioni.
- **MINOR** — nuove funzioni compatibili (es. nuova opzione backup).
- **MAJOR** — cambi formato backup, OAuth, o comportamento sync incompatibile.

8. Sistema supporto

Architettura

```
App Electron --POST JSON--> API Aven Labs --Resend--> Email interna + conferma cliente
```

- URL: `https://aven-labs.com/api/support/easyfatt-sync` (`supportConstants.js`).
- Timeout client: **30 secondi** (`support.js`).

Payload (campi principali)

Campo	Note
<code>name</code> , <code>email</code> , <code>message</code>	Obbligatori (validazione)

Campo	Note
phone	Opzionale
issueType	Etichetta italiana da <code>supportIssueTypes.js</code>
appVersion , platform , platformLabel	Contesto tecnico
lastSyncAt , lastSyncRows , googleAuthorized	Diagnostica
excelPath , sheetName	Config (attenzione privacy in email interna)

Backend di esempio

Cartella: `backend-examples/nextjs/`

- Route: `app/api/support/easyfatt-sync/route.ts`
- Template email: `lib/support-email-templates.ts`
- Validazione server: `lib/support-validation.ts`

Variabili env tipiche:

```
RESEND_API_KEY=re_...
SUPPORT_FROM_EMAIL="Aven Labs <support@aven-labs.com>"
SUPPORT_TO_EMAIL="support@aven-labs.com"
```

Email cliente

Conferma automatica all'indirizzo inserito nel form, con footer privacy (link policy/termini).

9. Backup e ripristino

Formato file JSON

Estensione consigliata: `.easyfatt-sync-backup.json`

Nome manuale tipico: `easyfatt-sync-backup-YYYY-MM-DD.json`

Nome automatico: `easyfatt-sync-backup-YYYY-MM-DD-HH-mm.json`

Struttura payload

```
{
  "app": "Easyfatt Sync",
  "backupVersion": "1.0",
  "backupType": "manual",
  "createdAt": "2026-05-16T20:00:00.000Z",
  "appVersion": "1.0.0",
  "platform": "darwin",
  "config": { },
  "legalAccepted": true,
  "legalAcceptedAt": "2026-01-01T00:00:00.000Z",
  "legalVersion": "1.0",
  "googleTokenIncluded": false,
  "googleToken": null
}
```

Campo	Descrizione
<code>backupType</code>	<code>"manual"</code>
<code>config</code>	Oggetto completo impostazioni electron-store
<code>googleToken</code>	Presente solo se l'utente ha spuntato l'inclusione

Non incluso: file Excel, dati clienti nel foglio, log applicativi.

Restore

1. Validazione `app`, `backupVersion`, `config`.
2. Merge config con default (`getDefaultConfig`).
3. Ripristino flag legali.

4. Token Google: scrittura o logout locale.
5. `restartScheduler` + `config-updated` verso UI.

Backup automatici

Impostazione	Default
<code>automaticBackupEnabled</code>	<code>false</code>
<code>automaticBackupFrequency</code>	<code>daily</code>
<code>automaticBackupTime</code>	<code>20:00</code>
<code>automaticBackupFolder</code>	<code>" "</code>
<code>automaticBackupRetention</code>	<code>10</code>
<code>automaticBackupIncludeGoogleToken</code>	<code>false</code>

- Settimanale: lunedì all'orario indicato.
- Mensile: primo giorno del mese.
- Cleanup: mantiene ultimi N file con pattern automatico; **non** cancella backup manuali.

10. Sicurezza

Electron hardening

In `main.js` → `webPreferences` :

```
contextIsolation: true,  
nodeIntegration: false,  
sandbox: true,  
preload: path.join(__dirname, "preload.js"),
```

Preload sicuro

- Nessun accesso diretto a `fs` , `process` , moduli Google.
- Solo metodi espliciti su `easyfattSync` .
- `open-external` valida URL (`^https?://`).

Token e credenziali

Asset	Posizione	Git
OAuth client secret	<code>oauth_credentials.json</code>	Ignorato
Refresh/access token	<code>EasyfattSync/token.json</code>	Ignorato
Config utente	<code>electron-store (userData)</code>	Locale

API backend

- L'app non incorpora API key Resend.
- Il supporto passa da HTTPS verso dominio Aven Labs.

Buone pratiche release

- Firmare binari Windows/macOS quando possibile (SmartScreen / Gatekeeper).
- Non allegare token o Excel cliente alle issue GitHub.

11. Workflow release

Checklist completa

- Aggiornare `version` in `package.json` (es. `1.0.1`).
- Aggiornare note di rilascio (CHANGELOG o body GitHub Release).
- Verificare `build.publish` (owner/repo corretti).

- Build:

```
npm run dist:win    # su Windows o CI Windows
npm run dist:mac    # su Mac ARM
npm run dist:mac:intel
```

- Controllare artefatti in `dist/` :
 - Installer `.exe` + `latest.yml` (Windows)
 - `.dmg` + `latest-mac.yml` o equivalente (macOS)
- Creare tag Git: `git tag v1.0.1 && git push origin v1.0.1`
- Creare **GitHub Release** dal tag.
- Caricare **tutti** gli asset generati (incluso `latest.yml`).
- Test su macchina pulita: install → avvio → check update → sync smoke test.
- Comunicare agli utenti (email/newsletter se previsto).

Publish con token

```
export GH_TOKEN=<github_pat_with_repo_scope>
npm run dist:win
# electron-builder può pubblicare se configurato --publish always
```

In CI è preferibile usare GitHub Actions con secret `GH_TOKEN` .

Compatibilità aggiornamenti

- Gli utenti su versione N ricevono update solo se `latest.yml` sulla release è coerente con la piattaforma installata.
- Non forzare downgrade: electron-updater installa versioni più recenti.

12. TODO / roadmap

Roadmap indicativa (non impegnativa):

Priorità	Funzione	Note
Alta	Sync bidirezionale	Sheets → Excel o merge bidirezionale con conflict resolution
Media	Multi-sheet	Più fogli Excel / più tab Sheets
Media	Sync prodotti / fatture	Estensione modello dati oltre anagrafica clienti
Bassa	Cloud dashboard	Stato sync centralizzato per rivenditori
Bassa	Analytics	Metriche sync, errori aggregati (rispetto privacy)
Bassa	Multi-utente	Account Aven Labs, config cloud, ruoli

Ogni voce richiede ADR, aggiornamento `backupVersion` se cambia schema, e revisione privacy.

13. Troubleshooting tecnico

OAuth

Sintomo	Causa probabile	Azione
<code>oauth_credentials.json</code> mancante	File non in root dev	Copiare credenziali Desktop da GCP
Redirect error	Tipo client errato	Usare Desktop app, non Web senza URI
<code>invalid_grant</code>	Token revocato/ scaduto	Eliminare <code>token.json</code> , ricollegare
Porta occupata	Server OAuth precedente	Riavviare app, chiudere processi Electron

Sync

Sintomo	Causa probabile	Azione
File in uso	Easyfatt tiene lock	Chiudere export; verificare retry in <code>sync.js</code>
0 righe	Foglio Excel vuoto o solo intestazione	Verificare export Easyfatt
Permission denied Sheets	Account sbagliato / ID errato	Controllare <code>spreadsheetId</code> e permessi foglio
Doppia sync	Race (raro post-mutex)	Verificare un solo job cron per orario

Updater

Sintomo	Causa probabile	Azione
"Solo versione installata"	<code>npm run dev</code>	Testare su build packaged
Update non trovato	<code>latest.yml</code> mancante in release	Allegare YAML alla GitHub Release
Download fallito	Rete / proxy aziendale	Log <code>electron-log</code> , test manuale URL release

Build

Sintomo	Piattaforma	Azione
NSIS fallisce	Windows	Path senza spazi, antivirus disabilitato per <code>dist/</code>
DMG non si apre	macOS	Gatekeeper: firmare e notarizzare per distribuzione esterna
	Tutte	Verificare <code>assets/icon.png</code>

Sintomo	Piattaforma	Azione
Icona mancante		

Permessi file

- Backup automatico: cartella deve essere scrivibile (`fs.accessSync W_OK`).
- Excel su rete SMB: latenza alta → aumentare debounce o disabilitare watch.

Token Google in backup

- Backup con token è sensibile: trattare file JSON come segreto.
- Dopo restore su altro PC, verificare che il refresh funzioni o ricollegare.

Contatti tecnici

- **Supporto prodotto:** support@aven-labs.com
- **Documentazione utente:** `[docs/client/README.md](../client/README.md)`

Ultimo aggiornamento documentazione: allineata a Easyfatt Sync v1.0.0 (stack Electron 42, backup v1.0).